

CEN350 Database Management Systems Final Homework

SOLUTIONS SET

1. The database schema that reflects all the constraints given in the ER diagram is as follows:

AUTHORS(SSN, Name, Address)

PUBLISHERS(Name, Address)

BOOKS(ISBN, Title, Amount, Year, Price, Pub_Name)

WRITE(SSN, ISBN)

The SQL commands are listed below:

```
CREATE TABLE Authors(SSN CHAR(11),
                      Name CHAR(50),
                      Address CHAR(200),
                      PRIMARY KEY (SSN));
```

```
CREATE TABLE Publishers(Name CHAR(250),
                          Address CHAR(200),
                          PRIMARY KEY (Name));
```

```
CREATE TABLE Books(ISBN CHAR(13),
                    Title CHAR(200),
                    Amount INTEGER,
                    Year INTEGER,
                    Price REAL,
                    Pub_Name CHAR(250) NOT NULL,
                    PRIMARY KEY (ISBN),
                    FOREIGN KEY Pub_Name REFERENCES Publishers(Name) ON DELETE NO ACTION);
```

```
CREATE TABLE Write(SSN CHAR(11),
                    ISBN CHAR(13),
                    PRIMARY KEY (ISBN, SSN),
                    FOREIGN KEY ISBN REFERENCES Books ON DELETE NO ACTION ON UPDATE CASCADE,
                    FOREIGN KEY SSN REFERENCES Authors ON DELETE NO ACTION ON UPDATE CASCADE);
```

```
CREATE ASSERTION AllBooksMustHaveAuthor(
    CHECK ( 0 = (SELECT COUNT(ISBN)
                  FROM Books
                  WHERE ISBN NOT IN (SELECT ISBN
                                     FROM Write))));
```

2. The set of functional dependencies is given as follows:

CustomerID $\rightarrow$ CustomerName
 BarCode $\rightarrow$ ProductName, Weight, Price
 CustomerID, BarCode $\rightarrow$ AmountBought

- a) What is the primary key of the above table is the combination of CustomerID and BarCode attributes. The closure of CustomerID and BarCode is computed as:

CustomerID BarCode $^+ = \{ \text{CustomerID, BarCode} \}$
 $\{ \text{CustomerID, CustomerName, BarCode} \}$ by using the 1st FD
 $\{ \text{CustomerID, CustomerName, BarCode, ProductName, Weight, Price} \}$ by using the 2nd FD
 $\{ \text{CustomerID, CustomerName, BarCode, ProductName, Weight, Price, AmountBought} \}$ by using the 3rd FD.

As CustomerID and BarCode together determine all the attributes in the table, CustomerID BarCode form the primary key.

- b) The above table is not in BCNF. Because, all the FDs are non-trivial, and in the 1st and 2nd FDs the left hand side do not contain any key.

CustomerID $\rightarrow$ CustomerName CustomerID is not a key. So, we decompose the table as follows:

Table 1 has the attributes : CustomerID, CustomerName. Therefore CustomerID becomes the primary key for this table.

Table 2 has the attributes: CustomerID, BarCode, ProductName, Weight, Price, AmountBought
 This table has the following FDs:

BarCode $\rightarrow$ ProductName, Weight, Price
 CustomerID, BarCode $\rightarrow$ AmountBought

BarCode is not a primary key, so the 2nd Table is not in BCNF. We need to decompose this table into two as follows:

Table 2.1 has the attributes: BarCode, ProductName, Weight, Price
 BarCode $\rightarrow$ ProductName, Weight, Price

BarCode is the key for this table, and it satisfies BCNF conditions.

Table 2.2 has the attributes: CustomerID, BarCode, AmountBought

CustomerID, BarCode $\rightarrow$ AmountBought

CustomerID BarCode combination is the key for this table, and it satisfies BCNF conditions.

3.

- a) Find the names of the students who enroll to courses that are enrolled by student whose name is “Mehmet Öz”.

SELECT S.Name

```

FROM Students S, Enroll E
WHERE S.StudentID = E.StudentID AND
      E.CourseID IN (SELECT E2.CourseID
                     FROM Students S2, Enroll E2
                     WHERE S2.StudentID = E2.StudentID AND
                           S2.Name LIKE "Mehmet Öz")

```

b) Find the names of the teachers who offer courses that are enrolled by "Mehmet Öz".

```

SELECT T.Name
FROM Teachers T, Offer O
WHERE T.SSN = O.SSN AND
      O.CourseID IN (SELECT E2.CourseID
                     FROM Students S2, Enroll E2
                     WHERE S2.StudentID = E2.StudentID AND
                           S2.Name LIKE "Mehmet Öz")

```

c) Find the names of courses that are enrolled by all "Computer Engineering" students.

```

SELECT C.CourseName
FROM Courses C
WHERE NOT EXISTS (SELECT S.StudentID
                  FROM Students S
                  WHERE NOT EXISTS (SELECT E.StudentID
                                    FROM Enroll E
                                    WHERE S.StudentID = E.StudentID AND
                                          E.CourseID = C.CourseID AND
                                          S.Department LIKE "Computer Engineering"))

```

d) Find the names of courses that are enrolled by some "Computer Engineering" students, and are not offered by "Ash Kaya".

```

SELECT C.CourseName
FROM Courses C, Enroll E, Students S
WHERE C.CourseID = E.CourseID AND
      E.StudentID = S.StudentID AND
      S.Department LIKE "Computer Engineering" AND
      C.CourseID NOT IN (SELECT O.CourseID
                        FROM Teachers T, Offer O
                        WHERE T.SSN = O.SSN AND
                              T.Name LIKE "Ash Kaya")

```

e) Find the names of the courses that are enrolled by at least 5 "Computer Engineering" students.

```

SELECT C.CourseName
FROM Courses C

```

```

WHERE C.CourseID IN (SELECT E.CourseID
                      FROM Enroll E, Students S
                      WHERE E.StudentID = S.StudentID AND
                           S.Department LIKE "Computer Engineering")
GROUP BY E.CourseID
HAVING COUNT(*) > 4)

```

4.

a) The problems in the given schedule are:

Unrepeatable read problem: T1 reads A, then T2 updates A, after that T1 reads A again. The value read by T1 has been changed.

Lost update problem: T2 updates B, then T1 updates B and commits. The update done by T2 is lost.

b) This schedule is re-written according to Strict 2 Phase Locking concurrency control protocol as follows:

T1	T2
S(A)	
R(A)	
S(B)	
R(B)	
	T2 has to wait for T1 to complete
X(B)	
W(B)	
R(A)	
commit	
	X(B)
	W(B)
	X(A)
	W(A)
	commit